IES Luis Braille – Coslada

Departamento de Informática

Titulación:	DAM	**Curso:**	2.º
Módulo:	Acceso a Datos	**Año:**	2026–2027
Examen:	RA1: Gestión de ficheros	**Fecha:**	_____
Duración:	85 min	**Nota:**	

Nombre y apellidos:	
RA:	RA1 (evaluación global)

Módulo: Acceso a Datos · 2.º DAM

Modalidad: Práctica individual en ordenador

Material permitido: IDE y autocompletado. **No se permite consultar apuntes.**

Puntuación total: 10 puntos.

1. Contexto y recursos proporcionados

Vas a desarrollar una pequeña aplicación que procesa ficheros de alumnos y genera nuevos ficheros de salida. El objetivo de la prueba es evaluar tu capacidad para **gestionar ficheros y directorios**, leer y escribir datos, y tratar errores sin interrumpir innecesariamente el proceso.

Se entrega un **proyecto Java preparado** que incluye:

- La clase `Alumno`, con los campos y métodos necesarios para consultar sus datos y ordenarlos según se especifique en el proyecto.
- Un método `parsear(String linea)` que transforma una línea CSV válida en un objeto `Alumno`. **No debes implementar ni modificar este método.**
- El código para pedir por consola la **ruta del directorio de salida**.
- Las clases y estructuras auxiliares que sean necesarias para que puedas centrarte en los ficheros.
- Un directorio de entrada con tres archivos:

Archivo	Contenido
`alumnos1.csv`	Un único alumno válido.
`alumnos2.csv`	200 alumnos válidos.
`alumnos3.csv`	Alumnos válidos, registros repetidos y líneas incorrectas.

Comportamiento del parseador: para cada línea, puede devolver un `Alumno`, lanzar una excepción por número incorrecto de campos, lanzar otra excepción por tipo de dato incorrecto o devolver `null` ante otros fallos. Los nombres exactos de las excepciones estarán definidos en el proyecto proporcionado.

Convenciones del examen:

- Se considera *alumno repetido* aquel que tenga el mismo **identificador único** que otro alumno ya incorporado. Se conserva la primera aparición y se descartan las posteriores. El identificador concreto se indica en la clase `Alumno` entregada.
- Cuando se solicite ordenar alumnos, se utilizará el **orden natural** definido en la clase `Alumno` del proyecto.
- Se utilizarán las rutas y los nombres de fichero indicados en este enunciado, salvo que el proyecto los proporcione mediante constantes.
- No se evaluará la implementación de la lógica de negocio ni del parseo CSV: ambos están resueltos en el código inicial.

2. Ejercicios

El método `main` ejecuta los ejercicios en orden y muestra una marca `[Flujo]` antes de cada uno y al terminar. Utiliza un directorio de salida nuevo en la primera ejecución; repite después con el mismo directorio para comprobar que el fichero final preexistente se conserva.

Ejercicio 1. Preparación del directorio de salida – 1 punto

La aplicación ya recibe por consola la ruta del directorio de salida.

Implementa la gestión de dicha ruta para que:

1. Compruebe si el directorio existe.
2. Si no existe, lo cree, incluidos los directorios intermedios que sean necesarios.
3. Si ya existe y es un directorio, no lo modifique.
4. Muestre siempre un mensaje adecuado indicando si se ha creado, si ya existía o si ha ocurrido un problema.
5. Si la ruta existe pero **no es un directorio**, comunique el error y no continúe escribiendo allí.

Ejercicio 2. Lectura de un alumno y escritura de texto – 1 punto

Lee `alumnos1.csv` utilizando el método `parsear(...)` proporcionado.

Crea en el directorio de salida un fichero llamado `alumno1.txt` que contenga **únicamente el nombre del alumno** seguido de un salto de línea. El valor debe obtenerse del objeto `Alumno`, no directamente de la cadena CSV.

Gestiona adecuadamente los recursos y los posibles errores de lectura y escritura.

Ejercicio 3. Lectura de una colección de alumnos – 2 puntos

Lee los datos de `alumnos2.csv` y guárdalos en una `List<Alumno>`.

Requisitos:

1. Utiliza una forma de lectura apropiada para un fichero de texto con varias líneas.
2. Procesa cada línea con el parseador proporcionado.
3. Gestiona correctamente los recursos abiertos y las excepciones que puedan producirse.
4. Muestra por consola cuántos alumnos se han incorporado a la lista.

Resultado esperado con el fichero facilitado: 200 alumnos en la lista.

Ejercicio 4. Escritura de un dato binario – 1 punto

Escribe en el directorio de salida un fichero llamado `numero_alumnos.bin` que contenga el **número de alumnos cargados desde** `alumnos2.csv`, guardado como **entero binario de 32 bits**, no como caracteres de texto.

Por ejemplo, escribir el texto `200` en un `.bin` **no cumple** el requisito. Gestiona el cierre del recurso de escritura.

Ejercicio 5. Fichero con errores y duplicados – 2 puntos

Lee `alumnos3.csv` y construye una nueva `List<Alumno>` con los alumnos válidos.

En este fichero encontrarás diferentes situaciones:

- Registros válidos.
- Registros repetidos **dentro de** `alumnos3.csv` (según el identificador único).
- Registros con menos o más campos de los esperados.
- Registros con un separador incorrecto.
- Registros con algún campo de tipo incorrecto.
- Registros para los que `parsear(...)` devuelve `null`.

Debes:

1. Incorporar únicamente los alumnos válidos y no repetidos a la lista de este ejercicio.
2. Ignorar las líneas incorrectas y los duplicados **sin detener la lectura de las siguientes líneas**.
3. Tratar de forma diferenciada las **dos excepciones** documentadas del parseador y el caso `null`.
4. Informar por consola del número de línea y del motivo por el que se descarta cada registro (incluidos los duplicados).
5. Mostrar al final el número de alumnos válidos incorporados.

No debes modificar el parseador para solucionar los errores del CSV.

Ejercicio 6. Integración y generación del fichero final – 3 puntos

Trabaja con las listas obtenidas en los ejercicios 3 y 5, y con el alumno del ejercicio 2.

Parte A. Primera generación

1. Combina las listas de `alumnos2.csv` y `alumnos3.csv`.
2. Elimina los alumnos repetidos por identificador único entre ambas listas, conservando el primero incorporado (primero los del fichero 2 y después los del 3).
3. Ordena la colección resultante según el orden natural definido en `Alumno`.
4. Crea un fichero de texto `alumnos_ordenados.csv` en el directorio de salida y escribe la colección ordenada, un alumno por línea, con el formato CSV que proporciona el proyecto.
5. **La creación debe hacerse con** `StandardOpenOption.CREATE_NEW`: si el fichero ya existe, debes informar de ello y **no modificarlo**.

Parte B. Actualización tras incorporar el alumno 1

6. **Solo si la Parte A ha conseguido crear el fichero**, incorpora el alumno procedente de `alumnos1.csv` a la colección, salvo que su identificador ya esté presente.
7. Vuelve a ordenar la colección.

8. Escribe **de nuevo la colección completa** en el mismo `alumnos_ordenados.csv`, reemplazando su contenido anterior (no añadiendo una segunda copia al final).

Importante: `CREATE_NEW` se usa exclusivamente en la **primera creación**. Para la segunda escritura tendrás que abrir el fichero ya creado de un modo que permita **sobrescribir/truncar su contenido**. Si el fichero existía antes de comenzar la Parte A, se omite todo el ejercicio 6 y el fichero debe permanecer intacto.

3. Criterios generales de corrección

Además del resultado, se valorará dentro de cada ejercicio:

- Uso adecuado de las API de ficheros y directorios de Java.
- Gestión correcta de recursos (por ejemplo, `try-with-resources` cuando proceda).
- Tratamiento adecuado de errores, sin silenciarlos ni interrumpir un proceso por errores recuperables de una sola línea.
- Diferenciación correcta entre datos de texto y datos binarios.
- Cumplimiento de las condiciones de creación, no sobrescritura y actualización solicitadas.
- Código claro, sin trabajo innecesario ajeno a la gestión de ficheros.

Ejercicio	Puntuación
1. Directorio de salida	1
2. Fichero 1: texto	1
3. Fichero 2: lista	2
4. Escritura binaria	1
5. Fichero 3: errores y duplicados	2
6. Integración y fichero final	3
Total de los ejercicios	**10**

Anexo A. README del proyecto del alumno

Examen práctico RA1 – Acceso a Datos (2.º DAM)

Los ejercicios NO están resueltos.

Preparación

Nota: los métodos de los ejercicios contienen `TODO` y no realizan todavía las operaciones pedidas.

El ejercicio 1 devuelve inicialmente `false`, por lo que **la aplicación no continuará hasta completarlo**. Es intencionado.

El método `main` marca el inicio de cada ejercicio con `[Flujo]`. Para revisar el resultado, usa un directorio de salida nuevo y ejecuta una segunda vez con la misma ruta: el fichero final debe permanecer intacto.

Archivos entregados

- `datos/entrada/alumnos1.csv` – 1 alumno correcto.
- `datos/entrada/alumnos2.csv` – 200 alumnos correctos.
- `datos/entrada/alumnos3.csv` – alumnos correctos, identificadores duplicados y registros mal formados.
- `src/main/java/.../Alumno.java` – clase de modelo con getters, `compareTo()` y `toCsv()`.
- `src/main/java/.../ParseadorAlumnos.java` – parser proporcionado.
- `src/main/java/.../NumeroCamposIncorrectoException.java` – excepción proporcionada.
- `src/main/java/.../TipoDatoIncorrectoException.java` – excepción proporcionada.
- `src/main/java/.../ExamenRA1.java` – **único fichero en el que debes completar los métodos TODO**.

Formato CSV

Cada línea (sin fila de cabecera) usa:

`id;nombre;apellidos;edad;notaMedia`

Ejemplo: `3001;Lucía;Sánchez Morales;21;8.60`

El separador es **punto y coma** (`;`) y las notas decimales se expresan con **punto** (`.`).

- Un ID identifica de forma única a cada alumno.
- `Alumno` tiene orden natural por apellidos, después nombre y después ID.
- `ParseadorAlumnos.parsear(linea)` puede devolver un `Alumno`, lanzar `NumeroCamposIncorrectoException`, lanzar `TipoDatoIncorrectoException` o devolver `null`.
- Los recursos de entrada no tienen fila de cabecera.

Restricciones

No se modifican los CSV, el parser, el modelo ni las excepciones. Se implementan exclusivamente los métodos indicados en `ExamenRA1.java`.

El fichero `alumnos_ordenados.csv` se crea con `CREATE_NEW` por primera vez y **solo si se ha creado en esta ejecución** se vuelve a escribir añadiendo el alumno 1. Si ya existía antes, debe mantenerse intacto.

Referencias Java del código proporcionado

Estos nombres pertenecen al proyecto inicial. No debes implementar ni modificar el modelo ni el parseador. El siguiente bloque recoge las llamadas disponibles; no constituye una solución a los ejercicios.

```
// Parseador proporcionado: puede lanzar las dos excepciones
// documentadas o devolver null ante otros fallos.
ParseadorAlumnos.parsear(linea);

// Métodos del modelo proporcionado.
alumno.getId();       // Identificador único.
alumno.getNombre();   // Nombre para alumno1.txt.
alumno.toCsv();       // Representación CSV para la salida.
```

Referencia de evaluación

RA1 (texto normativo): Desarrolla aplicaciones que gestionan información almacenada en ficheros identificando el campo de aplicación de los mismos y utilizando clases específicas.
Se evalúa globalmente el RA1. Las puntuaciones de los ejercicios son decisiones docentes.